

Stream & Task Basics

1. Create a source table `orders_source` with columns (`order_id` INT, `customer_id` INT, `amount` FLOAT, `order_date` DATE).

Create a stream on `orders_source` to capture all changes.

Answer :

Create source table:

```
1 CREATE OR REPLACE TABLE orders_source (  
2   order_id INT,  
3   customer_id INT,  
4   amount FLOAT,  
5   order_date DATE  
6 );  
7
```

Results (just now)	
Table	Chart
1 row 186ms	
status	
1	Table ORDERS_SOURCE successfully created.

Create stream to capture all changes:

```
8 CREATE OR REPLACE STREAM orders_source_stream  
9 ON TABLE orders_source;  
10
```

Results (just now)	
Table	Chart
1 row 235ms	
status	
1	Stream ORDERS_SOURCE_STREAM successfully created.

2. Create a target table `orders_target` with the same structure as `orders_source`.
Write a task that runs every 5 minutes to merge changes from the stream into `orders_target`.

Answer :

Create target table:

```
1 CREATE OR REPLACE TABLE orders_target (  
2   order_id INT,  
3   customer_id INT,  
4   amount FLOAT,  
5   order_date DATE  
6 );  
7
```

Results (just now)	
Table	Chart
1 row 136ms	
status	
1	Table ORDERS_TARGET successfully created.

Create task (runs every 30 sec):

```
18 CREATE OR REPLACE TASK orders_merge_task
19 WAREHOUSE = COMPUTE_WH
20 SCHEDULE = '30 second'
21 AS
22 MERGE INTO orders_target t
23 USING orders_source_stream s
24 ON t.order_id = s.order_id
25
26 WHEN MATCHED AND s.METADATA$ACTION = 'DELETE' THEN
27     DELETE
28
29 WHEN MATCHED THEN
30     UPDATE SET
31         t.customer_id = s.customer_id,
32         t.amount = s.amount,
33         t.order_date = s.order_date
34
35 WHEN NOT MATCHED AND s.METADATA$ACTION = 'INSERT' THEN
36     INSERT (order_id, customer_id, amount, order_date)
37     VALUES (s.order_id, s.customer_id, s.amount, s.order_date);
```

Results (just now)

Table Chart 1 row 76ms

status
Task ORDERS_MERGE_TASK successfully created.

ALTER TASK orders_merge_task RESUME;

```
38
39 ALTER TASK orders_merge_task RESUME;
40
```

Results (just now)

Table Chart 1 row 129ms

status
Statement executed successfully.

3. Insert sample data into orders_source and demonstrate how the stream captures these changes.

Show the effect of running the task on the orders_target table.

Answer :

Insert data into source table:

```
42 INSERT INTO orders_source VALUES
43 (1, 101, 250.50, '2026-01-01'),
44 (2, 102, 450.00, '2026-01-02');
45 INSERT INTO orders_source VALUES
46 (3, 103, 700.00, '2026-01-03');
47
```

Results (just now)

Table Chart 1 row 274ms

number of rows inserted
1

Check stream data (before task runs):

```
47
48
49 SELECT
50     order_id,
51     customer_id,
52     amount,
53     order_date,
54     METADATA$ACTION,
55     METADATA$ISUPDATE
56 FROM orders_source_stream;
```

Results (1 minute ago)

	ORDER_ID	CUSTOMER_ID	AMOUNT	ORDER_DATE	METADATA\$ACTION	METADATA\$ISUPDATE
1	1	101	250.5	2026-01-01	INSERT	FALSE
2	2	102	450	2026-01-02	INSERT	FALSE
3	3	103	700	2026-01-03	INSERT	FALSE

Traditional Batch & Multiple Tasks

4. You have three tables:

- staging_sales(id INT, product_id INT, sale_amount FLOAT)
- clean_sales(id INT, product_id INT, sale_amount FLOAT)
- sales_summary(product_id INT, total_sales FLOAT)

Answer:

```
61
62
63 create or replace table staging_sales(id INT, product_id INT, sale_amount FLOAT);
64 create or replace table clean_sales(id INT, product_id INT, sale_amount FLOAT);
65 create or replace table sales_summary(product_id INT, total_sales FLOAT);
```

Results (just now)

	status
1	Table SALES_SUMMARY successfully created.

Write SQL queries for three tasks that:

- **Task 1: Load raw data into staging_sales from an internal stage.**

```
66
67
68 CREATE OR REPLACE TASK task_load_staging_sales
69 WAREHOUSE = COMPUTE_WH
70 SCHEDULE = 'USING CRON 0 1 * * * Asia/Kolkata'
71 AS
72 COPY INTO staging_sales
73 FROM @internal_sales_stage
74 FILE_FORMAT = (TYPE = CSV);
```

sults (just now)

	status
1	Task TASK_LOAD_STAGING_SALES successfully created.

- Loads raw files from internal stage
- Copies data into staging_sales
- Runs daily at **1 AM**

- **Task 2: Transform and insert new records from staging_sales to clean_sales.**

```

75 CREATE OR REPLACE TASK task_load_clean_sales
76 WAREHOUSE = COMPUTE_WH
77 AFTER task_load_staging_sales
78 AS
79 INSERT INTO clean_sales (id, product_id, sale_amount)
80 SELECT
81     id,
82     product_id,
83     sale_amount
84 FROM staging_sales;

```

Results (just now)

Table Chart 1 row 112ms

id	status
1	Task TASK_LOAD_CLEAN_SALES successfully created.

- Runs **after Task 1**
- Moves data from staging to clean layer
- Represents **ETL transformation step**

- **Task 3: Aggregate data from clean_sales into sales_summary.**

```

86 CREATE OR REPLACE TASK task_load_sales_summary
87 WAREHOUSE = COMPUTE_WH
88 AFTER task_load_clean_sales
89 AS
90 MERGE INTO sales_summary tgt
91 USING (
92     SELECT
93         product_id,
94         SUM(sale_amount) AS total_sales
95     FROM clean_sales
96     GROUP BY product_id
97 ) src
98 ON tgt.product_id = src.product_id
99 WHEN MATCHED THEN
100     UPDATE SET total_sales = src.total_sales
101 WHEN NOT MATCHED THEN
102     INSERT (product_id, total_sales)
103     VALUES (src.product_id, src.total_sales);
104

```

Results (just now)

Table Chart 1 row 153ms

id	status
1	Task TASK_LOAD_SALES_SUMMARY successfully created.

- Aggregates sales per product
- Uses MERGE to handle existing & new products
- Represents **reporting / summary layer**

5. Schedule these tasks to run sequentially every day at 1 AM, 2 AM, and 3 AM respectively.

Task 1 – Runs daily at 1 AM

```

10 CREATE OR REPLACE TASK task_1_load_staging
11 WAREHOUSE = COMPUTE_WH
12 SCHEDULE = 'USING CRON 0 1 * * * Asia/Kolkata'
13 AS
14 COPY INTO staging_sales
15 FROM @internal_sales_stage
16 FILE_FORMAT = (TYPE = CSV);
17

```

Results (just now)

Table	Chart	1 row	88ms	Download	Refresh
status					
1		Task TASK_1_LOAD_STAGING successfully created.			

Task 2 – Runs daily at 2 AM

```

118 CREATE OR REPLACE TASK task_2_load_clean
119 WAREHOUSE = COMPUTE_WH
120 SCHEDULE = 'USING CRON 0 2 * * * Asia/Kolkata'
121 AS
122 INSERT INTO clean_sales
123 SELECT id, product_id, sale_amount
124 FROM staging_sales;
125

```

Results (just now)

Table	Chart	1 row	108ms	Download	Refresh
status					
1		Task TASK_2_LOAD_CLEAN successfully created.			

Task 3 – Runs daily at 3 AM

```

126 CREATE OR REPLACE TASK task_3_sales_summary
127 WAREHOUSE = COMPUTE_WH
128 SCHEDULE = 'USING CRON 0 3 * * * Asia/Kolkata'
129 AS
130 INSERT INTO sales_summary
131 SELECT product_id, SUM(sale_amount)
132 FROM clean_sales
133 GROUP BY product_id;

```

Results (just now)

Table	Chart	1 row	70ms	Download	Refresh
status					
1		Task TASK_3_SALES_SUMMARY successfully created.			

After Enable the tasks:

```

135 ALTER TASK task_1_load_staging RESUME;
136 ALTER TASK task_2_load_clean RESUME;
137 ALTER TASK task_3_sales_summary RESUME;
138

```

Results (just now)

Table	Chart	1 row	90ms	Download	Refresh
status					
1		Statement executed successfully.			

Three Snowflake tasks are scheduled using CRON to run daily at 1 AM, 2 AM, and 3 AM respectively for sequential batch processing.

SCD1 Implementation

6. Create tables:

- employee_source(emp_id INT, emp_name STRING, department STRING, salary FLOAT)
- employee_target(emp_id INT, emp_name STRING, department STRING, salary FLOAT)
- Write a stream on employee_source and a task to merge changes into employee_target implementing SCD Type 1 logic (overwrite existing data).

Answer:

Create Source Table :

```
117 CREATE OR REPLACE TABLE employee_source (  
118     emp_id INT,  
119     emp_name STRING,  
120     department STRING,  
121     salary FLOAT  
122 );  
123
```

Results (just now)

Table Chart

1 row 165ms

	status
1	Table EMPLOYEE_SOURCE successfully created.

Create Target Table :

```
125 CREATE OR REPLACE TABLE employee_target (  
126     emp_id INT,  
127     emp_name STRING,  
128     department STRING,  
129     salary FLOAT  
130 );  
131
```

Results (just now)

Table Chart

1 row 182ms

	status
1	Table EMPLOYEE_TARGET successfully created.

Create Stream on Source Table : (The stream captures INSERT, UPDATE, DELETE changes from employee_source)

```
132 CREATE OR REPLACE STREAM employee_stream  
133 ON TABLE employee_source;  
134
```

Results (just now)

Table Chart

1 row 618ms

	status
1	Stream EMPLOYEE_STREAM successfully created.

Create Task to Implement SCD Type 1 (MERGE) : This task overwrites existing records in the target table.

```
135 CREATE OR REPLACE TASK employee_scd1_task
136 WAREHOUSE = COMPUTE_WH
137 SCHEDULE = '5 MINUTE'
138 AS
139 MERGE INTO employee_target tgt
140 USING employee_stream src
141 ON tgt.emp_id = src.emp_id
142
143 -- Update existing record (overwrite old data)
144 WHEN MATCHED AND src.METADATA$ACTION IN ('INSERT', 'UPDATE') THEN
145 UPDATE SET
146     tgt.emp_name = src.emp_name,
147     tgt.department = src.department,
148     tgt.salary = src.salary
149
150 -- Insert new record
151 WHEN NOT MATCHED AND src.METADATA$ACTION = 'INSERT' THEN
152 INSERT (emp_id, emp_name, department, salary)
153 VALUES (src.emp_id, src.emp_name, src.department, src.salary)
154
155 -- Handle delete from source (optional but recommended)
156 WHEN MATCHED AND src.METADATA$ACTION = 'DELETE' THEN
157 DELETE;
```

Results (just now)

Table		Chart	1 row	93ms	⬇	⌚
1	Task EMPLOYEE_SCD1_TASK successfully created.					

Start the Task and Test with Sample Data : (After the task runs, data appears in employee_target.)

```
159 ALTER TASK employee_scd1_task RESUME;
160 INSERT INTO employee_source VALUES
161     {1, 'Ravi', 'IT', 60000},
162     {2, 'Anita', 'HR', 50000};
163
```

Results (just now)

Table		Chart	1 row	397ms	⬇	⌚
1	# number of rows inserted					
1						2

Update Source (SCD1 Overwrite Example) : Target table gets **overwritten** (old salary is lost).

```
164 UPDATE employee_source
165 SET salary = 70000, department = 'Engineering'
166 WHERE emp_id = 1;
```

Results (just now)

Table		Chart	1 row	392ms	⬇	⌚
1	# number of rows updated	# number of multi-joined rows updated				
1	1	0				

```
168 | select * from employee_source
```

```
169
```

Results (just now)

Table Chart 2 rows 188ms

	# EMP_ID	EMP_NAME	DEPARTMENT	# SALARY
1	1	Ravi	Engineering	70000
2	2	Anita	HR	50000

The final output is the employee_target table containing only the latest employee records, where updates overwrite existing data and no historical data is stored (SCD Type 1).

SCD2 Implementation

7. Create tables:

- product_source(product_id INT, product_name STRING, category STRING)
- product_target(product_id INT, product_name STRING, category STRING, start_date DATE, end_date DATE, is_current BOOLEAN)
- Write a stream and task that implements SCD Type 2 logic by tracking historical changes in product_target.

Answer :

Create Source Table :

```
172 | CREATE OR REPLACE TABLE product_source (
```

```
173 |     product_id INT,
```

```
174 |     product_name STRING,
```

```
175 |     category STRING
```

```
176 | );
```

```
177
```

Results (just now)

Table Chart 1 row 228ms

	status
1	Table PRODUCT_SOURCE successfully created.

Create Target Table (SCD Type 2) :

```
178 CREATE OR REPLACE TABLE product_target (  
179     product_id INT,  
180     product_name STRING,  
181     category STRING,  
182     start_date DATE,  
183     end_date DATE,  
184     is_current BOOLEAN  
185 );  
186
```

Results (just now)

Table Chart

1 row 192ms

status

1 Table PRODUCT_TARGET successfully created.

Create Stream on Source Table :

```
187 CREATE OR REPLACE STREAM product_stream  
188 ON TABLE product_source;  
189
```

Results (just now)

Table Chart

1 row 134ms

status

1 Stream PRODUCT_STREAM successfully created.

Create Task to Implement SCD Type 2 Logic

This task:

- Expires old record
- Inserts a new version
- Maintains history

```
190 CREATE OR REPLACE TASK product_scd2_task  
191 WAREHOUSE = COMPUTE_WH  
192 SCHEDULE = '5 MINUTE'  
193 AS  
194 MERGE INTO product_target tgt  
195 USING product_stream src  
196 ON tgt.product_id = src.product_id  
197 AND tgt.is_current = TRUE  
198  
199 -- Expire old record when date changes  
200 WHEN MATCHED  
201 AND src.METADATA$ACTION IN ('UPDATE')  
202 AND (  
203     tgt.product_name <> src.product_name  
204     OR tgt.category <> src.category  
205 )  
206 THEN UPDATE SET  
207     tgt.end_date = CURRENT_DATE - 1,  
208     tgt.is_current = FALSE  
209  
210 -- Insert new record  
211 WHEN NOT MATCHED  
212 AND src.METADATA$ACTION IN ('INSERT', 'UPDATE')  
213 THEN INSERT (  
214     product_id,  
215     product_name,  
216     category,  
217     start_date,  
218     end_date,  
219     is_current  
220 )  
221 VALUES (  
222     src.product_id,  
223     src.product_name,  
224     src.category,  
225     CURRENT_DATE,  
226     NULL,  
227     TRUE  
228 );  
229
```

Start the Task :

```
230 ALTER TASK product_scd2_task RESUME;
231
```

Results (just now)	
Table	Chart
1 row	128ms
status	
1	Statement executed successfully.

Example Data & Final Output :

```
232 INSERT INTO product_source VALUES
233 (101, 'Laptop', 'Electronics');
234
```

Results (just now)	
Table	Chart
1 row	235ms
# number of rows inserted	
1	1

Update Product Category :

```
235 UPDATE product_source
236 SET category = 'Computers'
237 WHERE product_id = 101;
238
```

Results (just now)	
Table	Chart
1 row	269ms
# number of rows updated	# number of multi-joined rows updated
1	0

239

select * from product_source;

Results (just now)

TableChart

🔍🔗2 rows101ms

	PRODUCT_ID	PRODUCT_NAME	CATEGORY
1	101	Laptop	Computers
2	101	Laptop	Electronics

The target table maintains full history by expiring old records and inserting new ones whenever a product changes, while always marking the latest record as current (SCD Type 2).

Truncate & Load Scenario

- Given the table daily_sales(date DATE, product_id INT, sales_amount FLOAT), write a task that truncates this table daily at midnight and loads fresh data from a stage.

Answer :

Scenario - We have a table : daily_sales(date DATE, product_id INT, sales_amount FLOAT)

Every day at midnight, you want to:

1. TRUNCATE the table (remove yesterday's data)
2. LOAD fresh data from a stage

Create Target Table :

```
242 CREATE OR REPLACE TABLE daily_sales (  
243   date DATE,  
244   product_id INT,  
245   sales_amount FLOAT  
246 );  
247
```

results (just now)

Table	Chart	1 row	164ms	⬇	🕒
🔍	📈	status			
1		Table DAILY_SALES successfully created.			

Create File Format (example: CSV) :

```
248 CREATE OR REPLACE FILE FORMAT sales_csv_format  
249 TYPE = CSV  
250 FIELD_DELIMITER = ','  
251 SKIP_HEADER = 1;  
252
```

results (just now)

Table	Chart	1 row	93ms	⬇	🕒
🔍	📈	status			
1		File format SALES_CSV_FORMAT successfully created.			

Create Stage :

```
253 CREATE OR REPLACE STAGE daily_sales_stage  
254 FILE_FORMAT = sales_csv_format;  
255
```

Results (just now)

Table	Chart	1 row	143ms	⬇	🕒
🔍	📈	status			
1		Stage area DAILY_SALES_STAGE successfully created.			

Create Task (Truncate & Load at Midnight) :

```
256 CREATE OR REPLACE TASK daily_sales_truncate_load_task  
257 WAREHOUSE = COMPUTE_WH  
258 SCHEDULE = 'USING CRON 0 0 * * * UTC'  
259 AS  
260 BEGIN  
261   -- Step 1: Truncate existing data  
262   TRUNCATE TABLE daily_sales;  
263   -- Step 2: Load fresh data  
264   COPY INTO daily_sales  
265   FROM @daily_sales_stage  
266   FILE_FORMAT = (FORMAT_NAME = sales_csv_format);  
267 END;  
268  
269
```

results (just now)

Table	Chart	1 row	65ms	⬇	🕒
🔍	📈	status			
1		Task DAILY_SALES_TRUNCATE_LOAD_TASK successfully created.			

Start the Task :

```
271 ALTER TASK daily_sales_truncate_load_task RESUME;  
272
```

Results (just now)

Table	Chart	1 row	107ms	⬇	🕒
🔍	📈	status			
1		Statement executed successfully.			

Final Output / Result

- Every day at 12:00 AM
- daily_sales table is fully cleared
- New data from the stage is loaded fresh
- No historical data is kept

Multiple Tasks with Dependencies

9. Write SQL to create two tasks:
- task_load_data: Loads data into raw_data table every hour.
 - task_process_data: Processes data in raw_data and loads into processed_data table, but only runs after task_load_data completes.

Answer :

- Task_load_data → runs every hour and loads data into **raw_data**
- Task_process_data → runs only after task_load_data completes successfully

Load data into raw_data (Hourly)

```
274 CREATE OR REPLACE TASK task_load_data
275 WAREHOUSE = COMPUTE_WH
276 SCHEDULE = 'USING CRON 0 * * * * UTC' -- every hour
277 AS
278 INSERT INTO raw_data
279 SELECT *
280 FROM @my_stage/raw_data_file;
```

results (just now)

Table Chart 1 row 249ms

id	status
1	Task TASK_LOAD_DATA successfully created.

Process data after Task 1 completes

```
282 CREATE OR REPLACE TASK task_process_data
283 WAREHOUSE = COMPUTE_WH
284 AFTER task_load_data
285 AS
286 INSERT INTO processed_data
287 SELECT
288     id,
289     UPPER(name) AS name,
290     amount
291 FROM raw_data;
```

results (just now)

Table Chart 1 row 97ms

id	status
1	Task TASK_PROCESS_DATA successfully created.

Enable (Resume) the Tasks



The screenshot shows a Snowflake SQL execution interface. At the top, two SQL statements are listed: `ALTER TASK task_load_data RESUME;` and `ALTER TASK task_process_data RESUME;`. Below the statements, the results are displayed in a table with the heading "results (just now)". The table has two columns: "id" and "status". The first row shows the statement executed successfully.

id	status
1	Statement executed successfully.

Simple Flow : `Task_load_data` → `task_process_data`

Bonus : Troubleshooting

10. Explain what could cause a task not to run even though it is scheduled and how to troubleshoot it.

Answer : A task may not run even if it is scheduled due to the following reasons:

- **Task is suspended** – Tasks are suspended by default. Resume it using `ALTER TASK ... RESUME`.
- **Parent task failed** – If the task depends on another task, it will run only when the parent task succeeds. Check task history and fix the parent task error.
- **The warehouse is not running** – Tasks need an active warehouse. Resume the warehouse.
- **Error in task SQL** – Syntax or data issues can stop the task. Run the SQL manually and fix the error.
- **Missing permissions** – Task role may not have required access. Grant needed privileges.
- **Wrong schedule / time zone** – Snowflake uses UTC time. Check CRON schedule and timing.